Laboratorio 10

Introducción - Recursividad

Un algoritmo recursivo es un procedimiento que resuelve un problema llamándose a sí mismo repetidamente con una versión más pequeña o simplificada de ese mismo problema, hasta alcanzar una condición de parada.

Todo algoritmo recursivo se compone de dos partes esenciales:

• Caso base: Condición que detiene la recursión al resolver el problema más pequeño posible sin hacer más llamadas (evita bucles infinitos).

• Caso recursivo: Parte de la función donde el algoritmo se llama a sí mismo con parámetros modificados (acercándose al caso base).

Ejercicios propuestos

Se solicita desarrollar los algoritmos de manera recursiva. Se piden en pseudocódigo y en python.

1. Para el arreglo de tamaño N y a través de algoritmos recursivos:
    a. Crear el arreglo con datos al azar
    b. Imprimir el contenido del arreglo
    c. Sumar el contenido del arreglo
    d. Determinar si un número es par o no
    e. Contar y sumar la cantidad de números pares dentro del arreglo.
    f. Contar y sumar la cantidad de número impares dentro del arreglo
    g. Ordenar el arreglo:
        i. Método de selección
        ii. Método de inserción
        iii. Método de burbuja
        iv. Método de shellsort
        v. Método de quicksort
    h. Implementar la búsqueda binaria.
2. Implementar las torres de Hanoi de manera recursiva.
3. “Subiendo escaleras”
   Una persona desea subir una escalera de N peldaños. En cada movimiento puede subir 1 o 2 peldaños.
   Escriba una función recursiva que determine de cuántas maneras distintas puede llegar a la cima.

Considere que el orden de los movimientos importa: subir primero 1 peldaño y luego 2 es una forma diferente de subir primero 2 y luego 1.

Entrada: Un número entero n, donde 1 ≤ n ≤ 45.

Salida: La cantidad de maneras distintas de subir la escalera.

Ejemplo 1:

- Entrada: n = 2

- Salida: 2

- Explicación: Las posibilidades son (1, 1) y (2).

Ejemplo 2:

- Entrada: n = 3

- Salida: 3

- Explicación: Las posibilidades son (1, 1, 1), (1, 2) y (2, 1).

Instrucciones

1. El laboratorio deberá ser desarrollado de manera **individual** utilizando Word y Python.
2. La solución:
   a. En pseudo código debe implementarse en un archivo word.
   b. En python deberá implementarse en un archivo con extensión **.py**, el cual deberá contener el algoritmo solicitado de acuerdo con los requisitos definidos en el laboratorio.
3. El programa deberá ejecutarse correctamente, sin presentar errores de compilación o ejecución, y cumplir con todas las funcionalidades especificadas para el algoritmo.
4. El código deberá seguir los principios de la programación estructurada, utilizando adecuadamente variables, funciones, estructuras de control y demás elementos vistos durante el curso.
5. Se deberán considerar los casos de prueba necesarios para verificar el correcto funcionamiento del programa, incluyendo la validación de las entradas cuando corresponda.
6. Utilizar como base de trabajo Pseudo código en el ANEXO

Evaluación.

1. La entrega del laboratorio será **individual** y deberá realizarse dentro del plazo establecido por el docente.

2. La evaluación se regirá por la **Pauta de Evaluación de Laboratorio** correspondiente a la asignatura, considerando tanto el correcto funcionamiento del programa como el cumplimiento de los requisitos establecidos.
3. Como parte de la evaluación, el estudiante deberá realizar una **defensa del código desarrollado**, en la cual deberá explicar la lógica de la solución implementada, el funcionamiento del algoritmo y las decisiones tomadas durante su desarrollo.
4. Durante la defensa, el docente podrá realizar preguntas relacionadas con el código presentado y solicitar modificaciones o explicaciones sobre partes específicas del programa, con el fin de verificar la comprensión del estudiante y la autoría del trabajo entregado.
5. Todos los trabajos serán evaluados en el laboratorio
6. **FECHA DE ENTREGA: 1 DE NOVIEMBRE DE 2026 HASTA LAS 24 HORAS.**

ANEXO

```
############################################################
BEGIN PRINCIPAL ORDENAR ARREGLO
        # N DEFINE EL TAMAÑO DEL ARREGLO
        DATOS[N]
        CREAR_ARREGLO(DATOS, N)
        IMPRIMIR_ARREGLO(DATOS, N)
        ORDENAR_ARREGLO(DATOS,N)
        # LLAMADO RECURSIVO SERÍA
        #ORDENAR_ARREGLO_RECURSIVO(DATOS, 1, 2, N)
        IMPRIMIR_ARREGLO(DATOS, N)
END
############################################################
```

```
############################################################
BEGIN ORDENAR_ARREGLO(DATOS,N)
        I = 1
        WHILE(I <N)DO
                J = I+1
                WHILE (J<=N) DO
                        IF (DATOS[I] < DATOS[J]) THEN
                                AUX = DATOS[I]
                                DATOS[I] = DATOS[J]
                                DATOS[J] =  AUX
                        ENDIF
                        J=J+1
                ENDWHILE
                I=I+1
        ENDWHILE
END
############################################################
```

```
#################################################################
BEGIN ORDENAR_ARREGLO_RECURSIVO(DATOS, I,J,N)
        IF(I <N)THEN
                IF (J<=N) THEN
                        IF (DATOS[I] < DATOS[J]) THEN
                                AUX = DATOS[I]
                                DATOS[I] = DATOS[J]
                                DATOS[J] =  AUX
                        ENDIF
                        ORDENAR_ARREGLO(DATOS, I, J+1, N)
                ELSE
                        ORDENAR_ARREGLO(DATOS, I+1, J, N)
                ENDIF
        ENDIF
END
#################################################################
```